
CHARACTERIZING PARALLELISM STRATEGIES IN LLM INFERENCE: FUNDAMENTAL COMPUTE-COMMUNICATION TRADE-OFFS

Javad Mirzaei*
 Office of CTO
 Dell Technologies Inc.
 Canada
 javad.mirzaei@dell.com

Jeebak Mitra
 Office of CTO
 Dell Technologies Inc.
 Canada
 jeebak.mitra@dell.com

ABSTRACT

Large Language Model (LLM) inference has become the dominant workload in modern AI systems, requiring serving infrastructures to simultaneously maximize throughput while satisfying stringent latency Service-Level Objectives (SLOs). Since state-of-the-art LLMs exceed the compute and memory capacity of a single GPU, inference is commonly distributed across multiple GPUs using tensor parallelism (TP), pipeline parallelism (PP), or hybrid parallelism (HB), which combines both approaches. However, selecting an appropriate parallelism strategy remains a challenging task because inference performance is governed by complex interactions among computation, inter-GPU communication, pipeline utilization, sequence length, batch size, and model architecture. Existing studies primarily rely on empirical evaluations or optimize specific serving components, offering limited analytical insights into the performance trade-offs among different parallelism strategies, particularly across the distinct prefill and decoding phases of inference.

In this paper, we present a unified analytical framework for modeling distributed LLM inference under TP, PP, and HB. Our framework decomposes end-to-end inference latency into computation, inter-GPU communication, and pipeline bubble overhead, and derives analytical models that explicitly characterize TP collective communication, PP point-to-point communication, and pipeline utilization as functions of hardware, model and workload characteristics. The proposed model further captures the fundamentally different execution characteristics of prefill and decoding, explaining why PP-dominant configurations favor compute-intensive prefill while TP configurations achieve lower decoding latency by eliminating pipeline bubbles. Extensive experiments using modern LLMs on multi-GPU platforms validate the model and confirm the fundamental trade-off between compute and communication of TP and PP across diverse workloads. Our framework provides practical insights for selecting efficient parallelism strategies and establishes a principled foundation for performance modeling and capacity planning in future LLM serving systems.

1 Introduction

Large Language Models (LLMs) have rapidly become a fundamental component of modern artificial intelligence (AI) applications, enabling conversational assistants, retrieval-augmented generation, code generation, and long-context reasoning systems. The rapid evolution of models from billions to hundreds of billions of parameters, together with the increasing adoption of long-context workloads, has introduced unprecedented computational and memory challenges during inference. Unlike model training, where large batches and long-running executions allow systems to maximize accelerator utilization, LLM serving must simultaneously achieve high throughput and satisfy strict latency requirements, including time-to-first-token (TTFT) and inter-token latency (ITL), under highly dynamic and heteroge-

*Corresponding author.

neous workloads [1, 2]. As a result, designing efficient inference systems has become a critical challenge for modern AI infrastructure.

To overcome the computational and memory limitations of large models, modern LLM serving systems increasingly rely on distributed execution across multiple GPUs. Tensor Parallelism (TP), introduced by Megatron-LM [3], partitions transformer computations across devices by sharding model weights and intermediate activations, reducing the computation and memory requirements per GPU. Pipeline Parallelism (PP), explored in systems such as GPipe [4] and PipeDream [5], partitions model layers across devices and enables models that exceed the memory capacity of a single accelerator. More recently, hybrid parallelism (HB), combining TP and PP, has been adopted in large-scale deployments to balance computation, memory capacity, and communication overhead [6, 7]. These parallelism strategies form the foundation of modern LLM inference frameworks, including TensorRT-LLM [8], vLLM [9], and NVIDIA Dynamo [10].

Despite extensive progress in distributed LLM serving, selecting an appropriate parallelism strategy remains challenging because performance depends on complex interactions among computation, communication, memory bandwidth, KV-cache capacity, and workload characteristics. Existing evaluations typically compare parallelism strategies through empirical measurements under fixed model configurations, batch sizes, and hardware settings. While such evaluations provide valuable insights, they often fail to explain the underlying mechanisms that determine performance scalability. In practice, the dominant bottleneck of LLM inference can shift significantly across workloads: prefill execution may be limited mainly by matrix computation, whereas decoding may become constrained by memory bandwidth, KV-cache accesses, and repeated synchronization overhead [1, 2]. Consequently, conclusions obtained from a single workload configuration often do not generalize to different models, sequence lengths, or deployment environments.

This challenge is further amplified by the fundamentally different characteristics of the prefill and decoding phases. During prefill, the system processes the entire input sequence simultaneously and benefits from large matrix operations with high arithmetic intensity. In contrast, decoding generates tokens sequentially, requiring repeated execution of the model for every generated token and introducing frequent communication and synchronization operations. For example, TP reduces computation per GPU but introduces collective communication after transformer components, while PP reduces communication volume but suffers from pipeline bubbles when the amount of useful work is insufficient to keep all stages occupied. Although recent serving systems employ advanced scheduling techniques, including continuous batching, KV-cache management, and phase-aware execution [1, 2, 11, 12], most systems still rely on a fixed parallelism configuration across prefill and decode phases. Therefore, understanding how parallelism overhead evolves across different inference phases remains an open challenge.

In this work, we address this challenge by developing a unified analytical framework for modeling distributed LLM inference performance. Unlike prior studies that focus on empirical benchmarking or isolated optimization techniques, our framework captures the fundamental latency components introduced by TP, PP, and HB across both prefill and decode execution. Specifically, we model TP communication overhead using collective communication costs, characterize pipeline bubble effects for autoregressive inference, and integrate computation, communication, and pipeline overhead into a unified performance model parameterized by model architecture, sequence length, batch size, GPU configuration, and interconnect topology. Our goal is to provide a fundamental understanding of the scalability limits of distributed LLM inference and enable systematic reasoning about the trade-offs among different parallelism strategies.

1.1 Related Works

To enable efficient execution of LLMs, prior research has extensively explored distributed parallelism strategies that partition computation and memory requirements across multiple accelerators. TP, introduced in Megatron-LM [3], distributes the computation of transformer layers by partitioning weight matrices across GPUs and synchronizing intermediate activations through collective communication operations. While TP effectively reduces per-device computation and enables high throughput within tightly coupled GPU systems, its scalability is limited by frequent communication overhead introduced by synchronization operations. PP, investigated in systems such as GPipe [4] and PipeDream [5], addresses memory scalability by partitioning model layers across sequential pipeline stages. However, PP introduces pipeline bubbles during pipeline fill and drain periods, reducing hardware utilization when the amount of available parallel work is insufficient. To overcome the limitations of TP and PP, HB strategies combining TP, PP, and Expert Parallelism (EP) have been proposed for large-scale model deployment [6, 7, 13]. Although these implementations provide effective mechanisms for scaling model execution, they primarily focus on improving system scalability and implementation efficiency rather than developing a unified understanding of the performance trade-offs among different parallelism strategies.

Beyond designing parallel execution techniques, analytical performance modeling has been extensively studied in high-performance computing and distributed deep learning to understand scalability limitations. Classical commu-

nication models, including the Hockney $\alpha - \beta$ model [14] and the LogP model [15], characterize communication latency as a combination of fixed startup overhead and bandwidth-dependent transmission cost. These models have been widely adopted for analyzing collective communication operations such as AllReduce and AllGather, including implementations provided by libraries such as NVIDIA collective communications library (NCCL) [16]. Similarly, the roofline model [17] provides a framework for identifying whether applications are limited by computation throughput or memory bandwidth. Recent distributed deep learning systems have extended these ideas to analyze communication and computation overlap in large-scale training workloads. However, existing analytical models are primarily designed for traditional HPC applications or distributed training scenarios and do not capture the unique characteristics of autoregressive LLM inference, where TP communication occurs repeatedly during decoding, pipeline bubbles depend on inference workload characteristics, and performance varies significantly between prefill and decode phases.

From serving perspectives, numerous techniques have been introduced, including continuous batching, KV-cache management, memory optimization, request scheduling, and workload-aware execution. Frameworks such as vLLM [2], TensorRT-LLM [8], Sarathi-Serve [1], DistServe [11], Splitwise [12], Mooncake [18], and NVIDIA Dynamo [10] have demonstrated significant improvements in practical LLM deployment by optimizing resource utilization and reducing serving latency. In parallel, several studies have proposed analytical approaches to model inference latency based on GPU utilization, memory bandwidth, KV-cache behavior, and workload characteristics. However, existing efforts generally focus on either improving serving efficiency empirically or modeling individual performance components in isolation. To the best of our knowledge, there is no unified analytical framework that jointly captures TP communication overhead, PP bubble effects, and their hybrid execution across both prefill and decode phases. This limitation prevents a fundamental understanding of how parallelism choices affect LLM inference scalability across different models, workloads, and hardware configurations, which motivates the analytical framework developed in this work backed by experiment results.

2 Parallelism Strategies

Existing LLM serving frameworks primarily employ three parallelism strategies: TP, PP, and HB, where HB combines TP and PP. Each strategy exposes a fundamentally different trade-off between computation, communication, and hardware utilization. TP partitions the computation of individual transformer layers across multiple GPUs, which enables concurrent execution at the cost of frequent collective communication and synchronization. In contrast, PP partitions the model layers across GPUs. It effectively reduces the inter-GPU communication to point-to-point activation transfers while introducing pipeline bubbles due to pipeline fill and drain effects. HB seeks to balance these trade-offs by combining TP within each pipeline stage and PP across stages, thereby reducing the limitations of either approach alone. Consequently, the relative efficacy of these strategies depends on multiple factors, including batch size, sequence length, model architecture, and GPU topology. This motivates a detailed analysis of their computation, communication, and pipeline characteristics. The following sections describe each parallelism strategy and develop analytical models that characterize their latency behavior during both the prefill and decoding phases.

2.1 Tensor Parallelism

Tensor Parallelism is one of the most widely adopted techniques for scaling LLM inference across multiple GPUs. As shown in Fig. 1, TP partitions the weight matrices of each transformer layer across multiple GPUs, allowing the computation of large matrix multiplications to be distributed among participating devices. This partitioning reduces the amount of work performed by each GPU and enables larger models to be served efficiently. However, because each GPU computes only a portion of the layer output, intermediate results must be exchanged and synchronized through collective communication operations. Consequently, TP introduces a fundamental trade-off: increasing the TP degree reduces computation per GPU but increases communication between GPUs. To better understand this trade-off, let us define the total latency in TP as

$$T_{\text{TP}}^{\text{total}} = T_{\text{TP}}^{\text{comp}} + T_{\text{TP}}^{\text{comm}}, \quad (1)$$

where $T_{\text{TP}}^{\text{comp}}$ and $T_{\text{TP}}^{\text{comm}}$ are the corresponding compute and communication terms, respectively. We further note that the compute latency for a transformer with L layers is $T_{\text{TP}}^{\text{comp}} = LT_{\text{TP}}^{(s),\text{comp}}$, where

$$T_{\text{TP}}^{(s),\text{comp}} = \frac{1}{P_{\text{TP}}} \frac{FLOPs_{\text{layer}}(B, S)}{\gamma} \quad (2)$$

is the compute latency in each layer, $FLOPs_{\text{layer}}(B, S)$ is the total floating-point operations required in each layer for a batch size B and sequence length S , P_{TP} is the TP degree, and γ denotes the effective compute throughput of a single GPU (FLOPs/s). The expression in (2) reflects the key property of TP, where partitioning model weights across more GPUs reduces the computation assigned to each GPU approximately in proportion to P_{TP} .

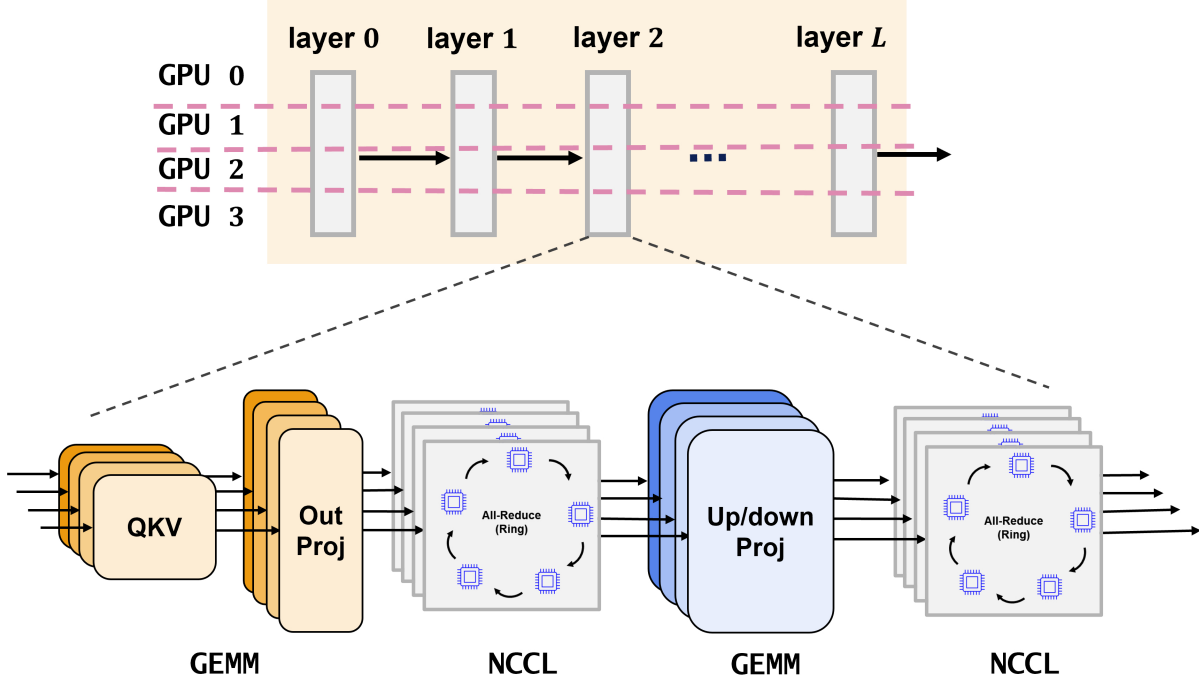


Figure 1: Tensor Parallelism Data Flow.

Communication Latency Modeling: NCCL [16] is the de facto communication backend in multi-GPU LLM serving. It provides highly optimized communication primitives over high-speed interconnects such as NVLink and InfiniBand. NCCL abstracts low-level communication procedures and automatically selects efficient communication algorithms based on the underlying hardware topology as well as the workload. NCCL organizes participating GPUs into a communicator, which defines the communication domain for collective operations. Before any communication can occur, a communicator is created using APIs such as `ncclCommInitAll` for single-process execution or `ncclCommInitRank` for distributed multi-process environments. NCCL also supports grouped execution through `ncclGroupStart` and `ncclGroupEnd`, allowing multiple communication operations to be launched together, thereby reducing kernel launch overhead and improving communication efficiency.

During inference, only a subset of NCCL primitives is utilized. In particular, `AllReduce` is the dominant communication primitive in Megatron-style TP [3], where each GPU computes partial activations that must be aggregated across all participating GPUs after the attention and feed-forward blocks of every transformer layer. Other collectives, including `ReduceScatter`, `AllGather`, `Broadcast`, and `Reduce`, are employed in different distributed training and inference frameworks, while `Send` and `Recv` are primarily used in PP for point-to-point activation transfers between adjacent pipeline stages [19]. Because transformer decoding performs these collective operations repeatedly for every layer and every generated token, the efficiency of inter-GPU communication directly affects the scalability of TP for inference. Consequently, accurately modeling communication latency is essential for understanding the trade-off between reduced computation and increased synchronization overhead as TP degree increases.

In Megatron-style TP [3], `AllReduce` is a dominant communication operation, where partial activations computed on different GPUs must be synchronized after the attention and feed-forward blocks of every transformer layer. Although TP reduces the computational workload assigned to each GPU, it introduces these recurring synchronization events that can significantly impact inference performance. Specifically, each transformer layer performs two activation synchronization steps, and these collective operations are executed for every layer and every generated token during decoding. As the TP degree increases, the amount of computation executed by each GPU decreases; while the synchronization overhead increases, which makes communication an increasingly important component of the overall latency budget. Our profiling results in Section 3.2 show that, despite the reduced per-GPU message size at higher TP degrees, inter-GPU communication can account for 30–40% of the end-to-end decoding latency. This behavior indicates that communication latency is governed not only by the volume of transmitted data but also by the frequency of

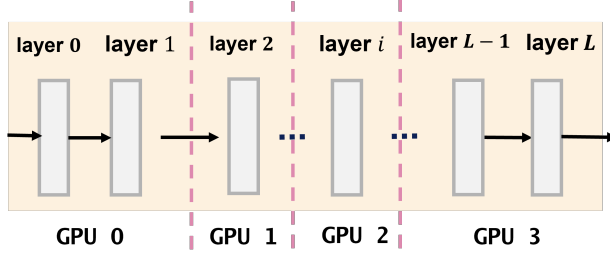


Figure 2: Pipeline Parallelism Data Flow.

collective operations and the latency characteristics of the underlying interconnect. To better understand this behavior and predict the scaling behavior of TP, we next develop an analytical model for inter-GPU communication latency that captures both message transmission costs and collective synchronization overhead. To this end, we define

$$|M| = BSd_{\text{hidden}}b, \quad (3)$$

the message size transferred between GPUs, $S = \text{ISL}$ for prefill, and $S = 1$ for decoding, d_{hidden} is the model’s hidden dimension, and b is the number of bytes for activations. We observe that the message size scales with batch size. Then, according to Megatron-style TP [3], the inter-GPU communication latency is given by:

$$T_{\text{TP}}^{\text{comm}} = LT_{\text{TP}}^{(s),\text{comm}} \quad (4)$$

where L is the number of transformer’s layers,

$$T_{\text{TP}}^{(s),\text{comm}} = k(P_{\text{TP}} - 1) \left[\alpha + \left(\frac{1}{P_{\text{TP}}} \right) \left(\frac{|M|}{\beta} \right) \right], \quad (5)$$

k is the number of collectives that occur per layer, α is the network’s intrinsic latency arising mainly from kernel launch and synchronization, and β is the effective inter-GPU NVLink bandwidth. This expression indicates that TP communication grows linearly with batch size and sequence length. Importantly, while increasing the TP degree reduces the communicated payload per GPU, it also increases the number of participating GPUs and the associated synchronization overhead.

The expressions in (2) and (5), expose the fundamental compute-communication trade-off in TP. In particular, computation decreases by P_{TP} , meanwhile, communication decreases only partially with message size while synchronization overhead increases with P_{TP} . This competition ultimately yields the optimal TP degree, to balance compute and communication in order to maximize throughput.

2.2 Pipeline Parallelism

Pipeline Parallelism adopts a fundamentally different approach from TP by partitioning the transformer layers across multiple GPUs rather than splitting individual layer computations. As shown in Fig. 2, each GPU is responsible for an L_{PP} contiguous subset of layers, and activations are forwarded from one stage to the next as execution progresses through the model. This approach significantly reduces the collective communication overhead associated with TP synchronization, as GPUs primarily exchange activations only between neighboring pipeline stages. Furthermore, PP enables serving larger models by distributing model weights across multiple devices. However, these benefits come at the cost of a performance inefficiency referred to as a pipeline bubble. Since different pipeline stages must wait for activations to arrive before computation can begin, some stages may remain idle during pipeline startup and drain phases, resulting in underutilized hardware resources. The magnitude of this idle time depends on both the pipeline depth and the number of batches available to keep the pipeline occupied. To quantify this idle latency, let us define the total latency of PP as

$$T_{\text{PP}}^{\text{total}} = T_{\text{PP}}^{\text{comp}} + T_{\text{PP}}^{\text{comm}} + T_{\text{PP}}^{\text{bubble}}, \quad (6)$$

where $T_{\text{PP}}^{\text{comp}}$, $T_{\text{PP}}^{\text{comm}}$, and $T_{\text{PP}}^{\text{bubble}}$ denote the compute, communication and bubble latency of PP, respectively. To quantify each component, we note that before all stages become active, the pipeline must be filled, which requires $P_{\text{PP}} - 1$ stage intervals. Similarly, after the final batch enters the pipeline, an additional $P_{\text{PP}} - 1$ stage intervals are required to drain the remaining work. Therefore, the total pipeline execution time can be modeled as

$$T_{\text{PP}}^{\text{total}} = (W + P_{\text{PP}} - 1)T_{\text{PP}}^{(s)}, \quad (7)$$

where $W = BS$ is the number of pipeline tasks (i.e., token-level work units) processed in one iteration ($S = \text{ISL}$ for prefill and $S = 1$ for decode), and

$$T_{\text{PP}}^{(s)} = T_{\text{PP}}^{(s),\text{comp}} + T_{\text{PP}}^{(s),\text{comm}}, \quad (8)$$

denotes the execution time of a single pipeline stage, including both computation and communication. Assuming the model is evenly partitioned across P_{PP} pipeline stages, each stage executes L/P_{PP} transformer layers. Given $FLOP_{s_{layer}}(B, S)$ as floating-point operations required by one transformer layer for batch size B and sequence length S , γ the effective computational throughput of a single GPU, the compute latency of one pipeline stage is given by

$$T_{PP}^{(s),comp} = \frac{L}{P_{PP}} \frac{FLOP_{s_{layer}}(B, S)}{\gamma}. \quad (9)$$

Then, PP's total compute latency, T_{PP}^{comp} , is given by

$$T_{PP}^{comp} = W T_{PP}^{(s),comp}. \quad (10)$$

From (10), compute latency increases with both the batch size and sequence length through $FLOP_{s_{layer}}(B, S)$. Therefore, prefill ($S = ISL$) incurs substantially higher computation than decoding ($S = 1$), while increasing the pipeline degree reduces the computation assigned to each stage approximately in proportion to $1/P_{PP}$.

To quantify the communication and bubble latency, we note that during inference, intermediate activations produced by one stage must be transmitted to the next stage before computation can proceed. Consequently, PP introduces inter-GPU point-to-point communication at every stage boundary. Unlike TP, PP does not shard activations across devices; therefore, $|M|$ scales linearly with both batch size and sequence length. Assuming a standard latency-bandwidth communication model, the communication latency between two adjacent stages can be expressed as

$$T_{PP}^{(s),comm} = \alpha + \frac{|M|}{\beta}. \quad (11)$$

Note that $T_{PP}^{(s),comm}$ scales proportionally with batch size and sequence length through $|M|$. Consequently, prefill requests with long input sequences ($S = ISL$) incur significantly higher communication costs than decode iterations ($S = 1$). Note that for small batch size, the activation size is relatively small, causing the startup latency term α to dominate the communication cost. In this regime, increasing the batch size improves communication efficiency by amortizing the fixed startup overhead across a larger activation payload. As the batch size continues to increase, the bandwidth term becomes dominant and communication latency grows approximately linearly with message size.

In addition to inter-GPU communication latency, PP suffers from pipeline bubbles arising from imperfect pipeline occupancy. In order to quantify T_{PP}^{bubble} , from (7), we note that only $W T_{PP}^{(s)}$ is the useful interval while the rest of $(P_{PP} - 1)T_{PP}^{(s)}$ is related to idle time (i.e. pipeline bubble). Therefore, the communication latency and pipeline bubble latency are given by

$$T_{PP}^{comm} = W T_{PP}^{(s),comm}, \quad (12)$$

$$T_{PP}^{bubble} = (P_{PP} - 1)(T_{PP}^{(s),comp} + T_{PP}^{(s),comm}), \quad (13)$$

and therefore, the bubble fraction is given by

$$\rho_{PP}^{bubble} = \frac{T_{PP}^{bubble}}{T_{PP}^{total}} = \frac{P_{PP} - 1}{W + P_{PP} - 1}. \quad (14)$$

From (13) and (14) we observe that T_{PP}^{bubble} increases with the PP degree, however ρ_{PP}^{bubble} decreases with batch size. Intuitively, the fill/drain overhead remains fixed at $P_{PP} - 1$ stage intervals, while increasing the number of batches allows this fixed cost to be amortized over a larger amount of useful work.

2.3 Hybrid Parallelism

In this section, we consider an HB configuration consisting of P_{PP} pipeline stages, each hosting L_{PP} consecutive layers of the model. The computational load of these layers is distributed among P_{TP} GPUs within each stage as described in Section 2.1. Therefore, the total number of GPUs used for parallelism is given by $P = P_{PP}P_{TP}$. For both prefill and decode, the total latency is given by

$$\begin{aligned} T_{HB}^{total} &= T_{HB}^{comp} + T_{PP}^{comm} + T_{PP}^{bubble}, \\ &= T_{TP}^{comp} + L_{PP} T_{TP}^{(s),comm} + T_{PP}^{comm} + T_{PP}^{bubble}, \end{aligned} \quad (15)$$

where T_{TP}^{comp} is the aggregated compute latency carried out by TP (see (4)), T_{PP}^{comm} denotes the communication latency between P_{PP} pipeline stages (see (12)), and T_{PP}^{bubble} is the bubble latency caused by inefficiencies in pipeline parallelism, which is described in more detail below.

Hybrid Parallelism Communication Modeling: During inference in HB, communication arises from two distinct sources. First, GPUs within the same TP group exchange partial activations through collective communication operations such as AllReduce, AllGather, and ReduceScatter. Second, activations are forwarded between adjacent PP stages through point-to-point communication. The per stage latency is defined as

$$T_{HB}^{(s)} = T_{HB}^{(s),comp} + T_{PP}^{(s),comm}. \quad (16)$$

Similar to PP, the total pipeline execution time is given by

$$T_{HB}^{total} = (W + P_{PP} - 1)T_{HB}^{(s)}, \quad (17)$$

and the bubble latency as

$$T_{HB}^{bubble} = (P_{PP} - 1)T_{HB}^{(s)}. \quad (18)$$

Therefore, the bubble fraction is given by

$$\rho_{HB}^{bubble} = \frac{T_{HB}^{bubble}}{T_{HB}^{total}} = \frac{P_{PP} - 1}{W + P_{PP} - 1}. \quad (19)$$

It is worth noting that (19) and (14) are identical. They are only a function of W , pipeline depth, and independent of TP degrees. In particular, TP changes $T_{HB}^{(s)}$ but not bubble latency. Furthermore, unlike TP, which only modifies the execution time of an individual pipeline stage through distributed computation and collective communication, the pipeline fill/drain process depends solely on the P_{PP} and the amount of work entering the pipeline. Consequently, introducing TP changes the absolute bubble latency through the stage execution time $T_{HB}^{(s)}$, but does not alter ρ_{HB}^{bubble} in (19). This implies that TP can reduce the duration of each pipeline interval, whereas improved pipeline utilization can be achieved via only increasing the amount of work W or reducing the pipeline depth P_{PP} .

Table 1: Sample Kernels.

Category	Kernel Name
GEMM	ampere_bf16_s16816gemm_bf16_256x128_ldg8_relu_f2f_stages_64x3_tn
	ampere_bf16_s16816gemm_bf16_64x64_ldg8_relu_f2f_stages_64x5_tn
	ampere_bf16_s16816gemm_bf16_256x128_ldg8_f2f_stages_64x3_tn
	ampere_bf16_s16816gemm_bf16_128x64_ldg8_f2f_stages_64x4_tn
	ampere_bf16_s16816gemm_bf16_64x64_sliced1x2_ldg8_f2f_stages_64x6_tn
	ampere_bf16_s16816gemm_bf16_64x64_sliced1x2_ldg8_relu_f2f_stages_64x5_tn
	ampere_bf16_s16816gemm_bf16_64x64_sliced1x2_ldg8_f2f_stages_64x5_tn
NCCL	ncclDevKernel_AllReduce_Sum_bf16_RING_LL
	ncclDevKernel_AllGather_RING_LL
	ncclDevKernel_SendRecv
	void vllm::cross_device_reduce_1stage
Attention	triton_poi_fused_cat_3
	triton_poi_fused_cat_4
	vllm::reshape_and_cache_flash_kernel
	flash::flash_fwd_splitkv_kernel
	flash::flash_fwd_splitkv_combine_kernel
Other	triton_red_fused__to_copy_add_mean_mul_pow_rsqr_2
	triton_red_fused__to_copy_add_mean_mul_pow_rsqr_0
	triton_poi_fused_mul_silu_1
	at::native::unrolled_elementwise_kernel
	at::native::index_elementwise_kernel
	Memcpy DtoH, Memcpy HtoD, Memcpy DtoD

3 Experiments

In this section, we provide experiments to evaluate modern LLMs deployed across multiple GPUs. The primary objective of our evaluation is to characterize the performance trade-offs among TP, PP, and HB during both prefill and decoding phases of autoregressive inference. We first describe the experimental setup, including the hardware platform, models, serving framework, and evaluation methodology. We then analyze TP, PP, and HB independently by

decomposing execution time into four major kernels: matrix multiplication (GEMM) operations, attention computation (Attention), communication overhead (NCCL), and miscellaneous operations (Other) such as kernel launches, memory management, and framework overhead. Samples of such kernels are provided in Table 1. For each parallelism strategy, we evaluate both TTFT and ITL across different batch sizes. The results in this section provide insights on how computation, communication, and pipeline utilization collectively determine the scalability and efficiency of distributed LLM inference.

3.1 Experiment Setup

Throughout the experiments, we employ Qwen2.5-32B-Instruct and Qwen2.5-14B-Instruct [20] on $8 \times$ NVIDIA A100 SXM4 (80 GB) [21]. The GPUs are connected through NVLink, providing an aggregate bidirectional bandwidth of 600 GB/s between neighboring GPUs, to enable high-throughput collective communication required by TP and PP for inference. Unless otherwise stated, inference is performed using FP16 precision with TP and PP degrees of 1,2,4,8. All experiments are performed using vLLM v0.15.0 as an inference engine [22].

We evaluate the efficiency of inference for both prefill and decoding across different batch sizes, while fixing the ISL and OSL. To characterize both prefill and decode performance, we report TTFT and ITL together with kernel-level execution breakdowns obtained through NVIDIA Nsight Systems [23], allowing us to separately quantify GEMM, Attention, NCCL, and remaining runtime overhead. Detailed experiment parameters are provided in Table 2.

Table 2: Experimental Parameters.

Parameter	Value
GPU Platform	$8 \times$ NVIDIA A100 SXM4 (80 GB)
GPU Memory	80 GB HBM2e per GPU
HBM Bandwidth	2.0 TB/s per GPU
Peak Compute	312 TFLOPS (FP16/BF16 Tensor Core)
Interconnect	NVLink 600 GB/s
Inference Framework	vllm v0.15.1
Model	Qwen2.5-32B-Instruct Qwen2.5-14B-Instruct
Precision	FP16
Parallelism Degrees	TP/PP $\in \{1, 2, 4, 8\}$
Batch Size	$B \in \{1, 2, 4, \dots, 128\}$
ISL	1000 tokens
OSL	200 tokens
Profiler	NVIDIA Nsight Systems v2026.5.1

3.2 Tensor Parallelism Scaling Characteristics

Communication Characterization: In Fig. 3, we plot T_{TP}^{comm} per token vs batch size for various TP degrees. For fixed TP, NCCL has three competing operating regions that govern the behavior of TP. At small batch sizes, and according to (5), the fixed communication latency term α dominates the overall communication cost. In this regime, the amount of data being exchanged is relatively small, and most of the communication time is spent initiating and coordinating collective operations rather than transferring payload data. As a result, increasing the batch size has only a limited impact on communication latency, since the fixed startup overhead is already the primary contributor. The system therefore operates in a latency-bound regime where communication cost is largely insensitive to message size.

As the batch size increases, the contribution of data transfer begins to grow and communication enters a transition region. In this regime, both α and the bandwidth-dependent transfer cost contribute meaningfully to T_{TP}^{comm} . Consequently, communication latency increases with batch size, but not yet in a strictly linear manner. The system is gradually shifting from being dominated by synchronization overhead toward being constrained by data movement. This region is particularly important because it often corresponds to the operating range of production serving systems, where both latency and throughput considerations must be balanced.

At sufficiently large batch sizes, α (i.e., startup latency) becomes negligible compared to the time required to move data across the interconnect. The communication cost is then dominated by the bandwidth term, causing latency to scale approximately linearly with message size. In this bandwidth-bound regime, further increases in batch size directly translate into higher communication overhead, and the achievable performance becomes limited by the effective inter-GPU bandwidth rather than synchronization latency. This observation highlights a key characteristic of

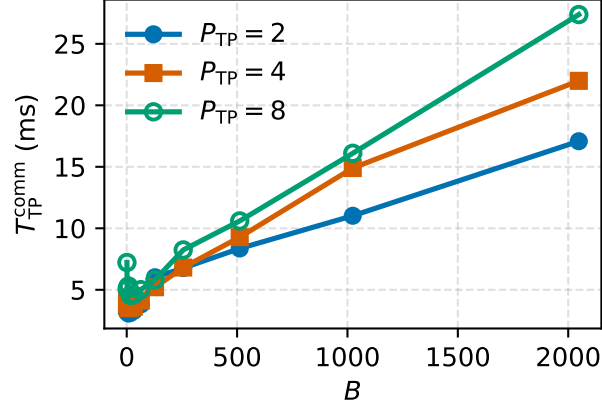


Figure 3: T_{TP}^{comm} per token vs. batch size for different TP degrees. TP communication is dominated by startup latency at small batch sizes and gradually transitions to bandwidth-limited behavior at large batch sizes.

TP, where small-batch workloads are primarily constrained by collective operation latency, large-batch workloads are fundamentally constrained by interconnect bandwidth, which makes efficient utilization of communication resources critical for sustaining scalability.

Effective Bandwidth: Figure 4a illustrates the achieved interconnect bandwidth during the prefill vs batch size for different TP degrees. Although the A100 NVLink interconnect provides a theoretical peak bandwidth of approximately 600 GB/s, the measured bandwidth never approaches this limit and instead saturates at roughly 250 GB/s. This behavior reflects the fact that collective communication performance is constrained by more than the raw link bandwidth. In particular, AllReduce operations require synchronization among all participating GPUs, kernel launch overhead, protocol processing, and reduction operations before data transmission can fully utilize the interconnect. During prefill, the communicated activation tensors are relatively large (i.e., $S = ISL$), allowing bandwidth utilization to increase rapidly with batch size. However, once the communication pipeline becomes saturated, additional increases in message size provide little improvement in effective bandwidth, resulting in the observed plateau. Interestingly, higher TP degrees consistently achieve higher bandwidth utilization. To explain this, we note that although the case where $P_{TP} = 8$ communicates fewer bytes per GPU due to tensor partitioning, the collective involves more participating GPUs and therefore generates a larger aggregate communication volume across the NVLink fabric. The communication subsystem consequently remains active longer to accommodate the communication among more GPU participants, amortizing startup costs and enabling the interconnect to operate closer to its sustainable throughput than the cases of $P_{TP} = 2$ and $P_{TP} = 4$.

During decoding, as shown in Fig.4b, the achieved bandwidth is substantially lower, saturating at only approximately 150 GB/s, or about 25% of the theoretical NVLink bandwidth. This reduction arises from the autoregressive nature of decoding, where only a single token is processed per iteration ($S = 1$). Consequently, each of AllReduce operation transfers significantly smaller messages, causing communication to remain dominated by startup latency and synchronization overhead rather than bulk data movement. The smaller message sizes also delay the transition into the bandwidth-bound regime, explaining why the bandwidth plateau occurs at much larger batch sizes than during prefill. As in the prefill phase, the case where $P_{TP} = 8$ consistently achieves higher bandwidth utilization than $P_{TP} = 4$ and $P_{TP} = 2$ because the increased number of GPUs participating in each collective produces greater aggregate network traffic and keeps the communication links busy for longer periods. Nevertheless, even the case $P_{TP} = 8$ remains far from saturating the available NVLink bandwidth, indicating that, in TP, decoding is fundamentally latency-limited rather than bandwidth-limited. This observation explains why increasing the TP degrees often increases the communication latency observed in Fig. 3, where adding more GPUs introduces additional synchronization costs without generating sufficiently large messages to fully exploit the available interconnect bandwidth.

Compute-Communication Trade-off: To evaluate the compute-communication trade-off, Fig. 5a shows the TTFT breakdown for different TP degrees. As TP degree increases from 1 to 8, the overall TTFT decreases because the computation performed by each GPU is reduced approximately proportionally with P_{TP} . During prefill, every input token participates in the forward pass through all transformer layers, making matrix multiplications kernels (GEMMs) the dominant contributor to latency. Partitioning matrix operations across more GPUs substantially reduces the computation assigned to each device, thereby resulting in a significant reduction in the GEMM component and consequently the overall TTFT. This trend is consistent with (2), where $T^{(s),comp}$ scales inversely with P_{TP} .

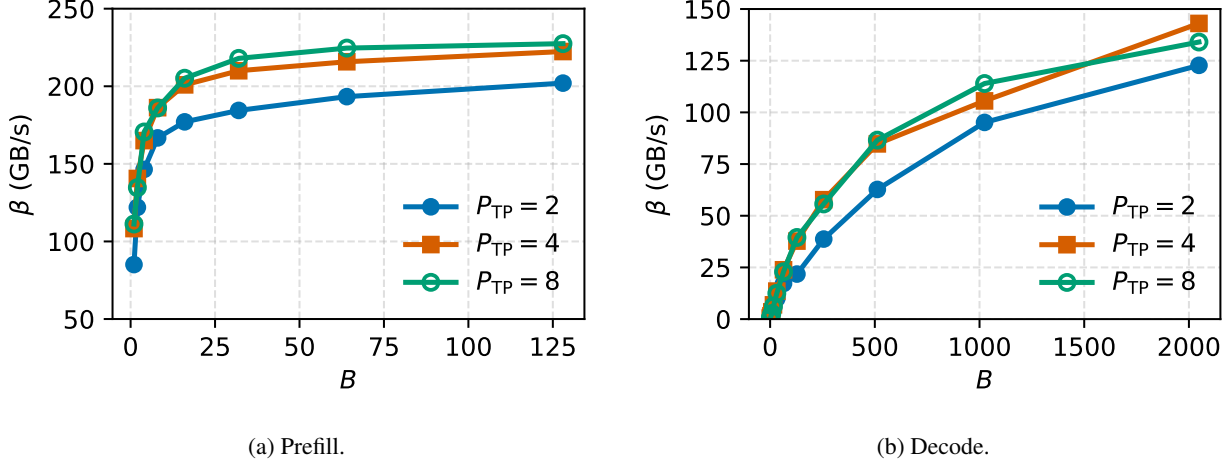


Figure 4: Effective bandwidth vs. batch size for different TP degrees during (a) Prefill and (b) Decode. Bandwidth utilization grows with communication volume and eventually plateaus, revealing the transition from latency-dominated communication to sustainable inter-connect throughput. Decode remains more latency-limited than prefill due to smaller message sizes, resulting in lower effective bandwidth utilization.

For fixed P_{TP} , TTFT increases almost linearly with batch size because the amount of computation grows proportionally with the number of processed tokens. During prefill, where $S = ISL$, both GEMM and NCCL increase with the total number of input tokens (i.e., BS). Consequently, both $T_{TP}^{(s),comp}$ and $T_{TP}^{(s),comm}$ increase with batch size. However, the GEMM component grows much faster than the NCCL component since prefill is primarily compute-bound. Although communication increases with the activation size, the computational complexity of GEMM scales much more rapidly. This makes GEMM the dominant contributor to TTFT over the evaluated batch sizes.

Fig. 5b presents the breakdown of ITL during decoding. Similar to TTFT, increasing P_{TP} reduces the overall ITL because each GPU performs only a fraction of the matrix multiplications. However, unlike prefill, the increase in ITL with batch size is much slower. During decoding, only a single token is generated per iteration ($S = 1$), meaning that both the computational workload and the communicated activation size grow only with the batch size. Consequently, each decoding iteration performs significantly less computation than prefill, leading to a much slower increase in both the GEMM and NCCL components. The communication overhead also increases more gradually because the activation tensors exchanged by NCCL are substantially smaller during decoding. According to (3), $|M|$ scales differently for both prefill and decode. As a result, the startup latency term α in (5) contributes a larger fraction of the communication time, causing NCCL latency to increase more slowly with batch size than during prefill.

It is worth noting that increasing P_{TP} has opposite effects on computation and communication. On one hand, partitioning the model weights across more GPUs reduces the amount of computation assigned to each GPU, causing the GEMM latency to decrease approximately proportionally with P_{TP} . On the other hand, TP requires synchronization after every attention and feed-forward block. Although the communicated payload per GPU decreases by P_{TP} , the number of participating GPUs and collective synchronization events increases with P_{TP} . Consequently, the startup latency and synchronization overhead become increasingly dominant. This causes the NCCL component to occupy a larger fraction of the total execution time. This behavior directly reflects the trade-off captured by the analytical model, where computation scales favorably with the P_{TP} , whereas communication exhibits diminishing returns due to the increasing cost of collective synchronization.

To understand this trade-off, in Fig. 6a, as we increase TP degree, TTFT decreases. the reason is that the time spent in GEMM kernels decreases substantially due to reduced per-GPU workload causing GEMM operations to decrease approximately in proportion to the degree of parallelism. At the same time, NCCL overhead grows steadily because more GPUs must participate in synchronization operations. Despite this increase in communication latency, the reduction in compute latency is initially larger than the communication penalty, resulting in an overall decrease in TTFT. In Fig. 6b, we show the contribution of each operation in TTFT. We observe that the total contribution of NCCL and GEMM remains the same across all TP degrees. However, as we increase TP degree, the communication overhead is traded for less GEMM operation per GPUs, thereby resulting a significant reduction in TTFT. Note that at higher TPs, the communication overhead consumes almost 40% of TTFT. A similar behavior is observed in ITL during decoding process as in Fig. 6c and 6d. We observe that TP successfully accelerates both prefill and decoding because the computational savings outweigh the additional communication overhead within the evaluated TP range.

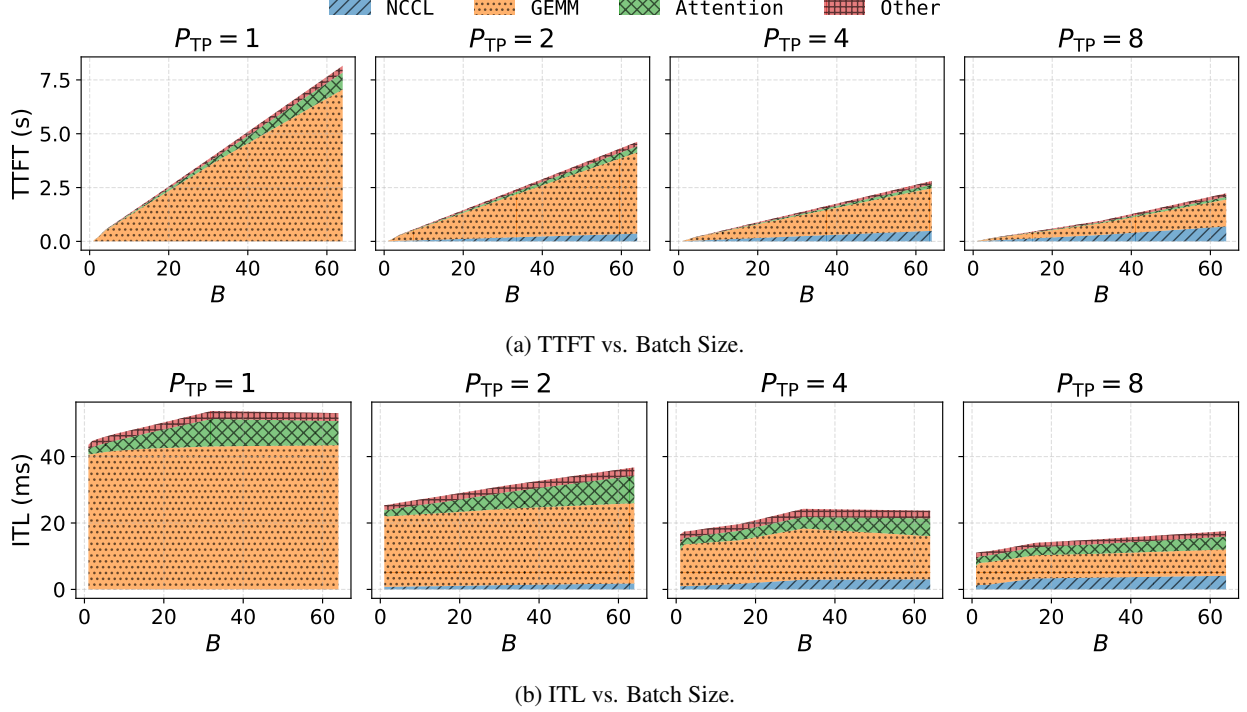


Figure 5: Kernel-level breakdown of (a) TTFT and (b) ITL across different TP degrees for Qwen2.5 - 32B - Instruct. There is a fundamental compute-communication trade-off in TP, where computational savings from distributed GEMM are partially offset by increased NCCL synchronization overhead.

In all the experiments in this section, we observed that, TP communication overhead originates from synchronization operations performed after distributed matrix multiplications. During these operations, each GPU exchanges partial results with all other participating GPUs through collective communication primitives. The latency of these communications depends on both the communication bandwidth available between GPUs and the number of synchronization events required throughout the model. As we increase TP degree, the frequency and cost of these communications become increasingly significant. Therefore, while GEMM kernels become faster due to workload partitioning, NCCL kernels consume a larger fraction of the total execution time. This behavior reflects a fundamental property of TP where reducing computation often requires additional coordination, and the benefits of parallel execution are ultimately constrained by communication efficiency.

The result in this section indicates that increasing TP does not eliminate bottlenecks; rather, it shifts them. At low TP degrees, inference execution is dominated by computation, particularly GEMM and Attention kernels, making the system primarily compute-bound. As additional GPUs are introduced, the computational burden is reduced and communication becomes increasingly prominent. Eventually, a growing fraction of execution time is spent waiting for synchronization rather than performing useful computation. This transition represents a bottleneck shift from computation-bound execution toward communication-bound execution. Understanding this bottleneck transition is essential for selecting an appropriate TP configuration, as the optimal TP degree depends not only on model size and available hardware, but also on the balance between computation and communication within the target workload.

3.3 Pipeline Utilization and Bubble Overhead

In PP, the latency is beyond the compute and communication inter-play. The pipeline inefficiencies introduces idle times (i.e., bubbles) that are particularly affect the latency during decoding. Since each active request contributes only a single token per decode iteration, the number of concurrently available work units is approximately equal to the active decode batch size. Consequently, for small decode batches $B \ll P_{PP}$, the pipeline remains underutilized and a substantial fraction of execution time is spent in fill/drain phases. As the decode batch size increases, the number of active batches also increases, allowing all pipeline stages to be occupied. In the limit of large batch sizes $B \gg P_{PP}$, the bubble fraction approaches zero. In contrast, bubble overhead is generally less significant during prefill. Although the number of batches may remain unchanged, prefill workloads involve substantially longer input sequences ($S = ISL$), resulting in much larger computational workloads per batch. Consequently, the fixed fill/drain overhead constitutes

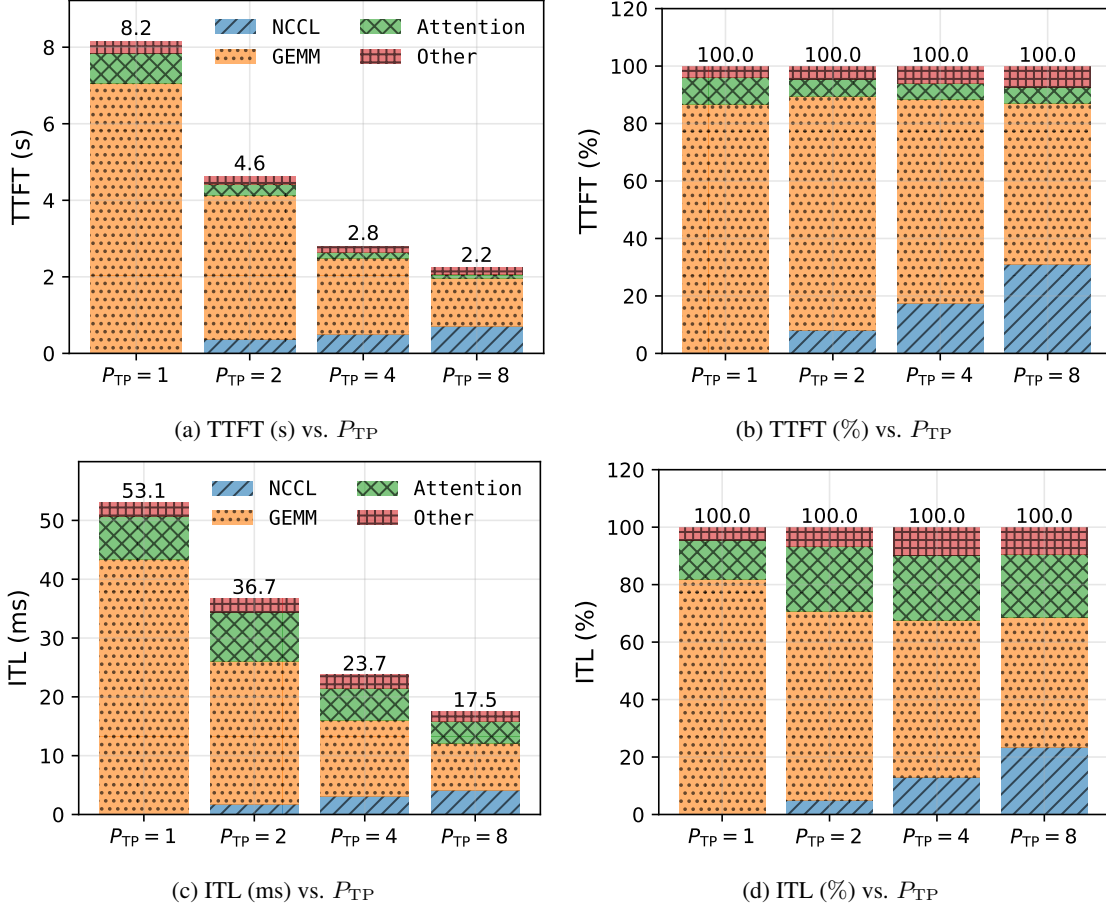


Figure 6: Actual and normalized kernel contributions to TTFT and ITL across various TP degrees for Qwen2.5-32B-Instruct and $B = 64$. Increasing TP degrees reduces both TTFT and ITL by lowering per-GPU computation, while NCCL synchronization consumes an increasing fraction of latency, illustrating the trade-off between compute reduction and communication overhead.

only a small fraction of the overall execution time, which makes pipeline bubbles largely negligible for long-context prefill workloads.

To understand the impact of pipeline bubbles during decoding, in Fig 7a, we analyze the ITL across different pipeline degrees and batch sizes. Our results show that for $P_{PP} > 1$, a significant fraction of the latency at small batch sizes is attributed to NCCL-related overhead. Unlike TP, these communication costs are not primarily caused by activation synchronization but rather reflect idle periods associated with pipeline bubbles. The effect becomes increasingly pronounced as the pipeline depth grows, since deeper pipelines require more stages to be filled before reaching steady-state execution. Consequently, the case $P_{PP} = 8$ exhibits substantially larger bubble overhead than the case $P_{PP} = 2$ or $P_{PP} = 4$ at low batch sizes. As batch size increases, the number of simultaneously active decoding requests grows, generating more in-flight tokens and therefore more batches that can be distributed across the pipeline stages. These additional batches progressively fill the pipeline, reducing idle time and amortizing the bubble cost over a larger amount of useful work. Therefore, the NCCL-dominated bubble overhead gradually diminishes.

The behavior during prefill is markedly different and can be observed through the TTFT breakdown in Fig 7b. Unlike decoding, where only a limited number of tokens are active at any given time, prefill processes the entire input sequence simultaneously. Long input sequences naturally generate a large number of micro-batches and activation waves that continuously occupy the pipeline stages throughout execution. Consequently, the pipeline remains close to full utilization for most of the prefill phase, leaving little opportunity for pipeline bubbles to emerge. This behavior is reflected in Fig 7b, where the NCCL component associated with bubble overhead is either significantly reduced or nearly absent across PP configurations. Instead, TTFT is primarily dominated by useful computation, including GEMM and attention kernels. These observations highlight a fundamental distinction between decoding and prefill workloads: while decoding efficiency is highly sensitive to pipeline utilization and the availability of in-flight tokens, prefill inher-

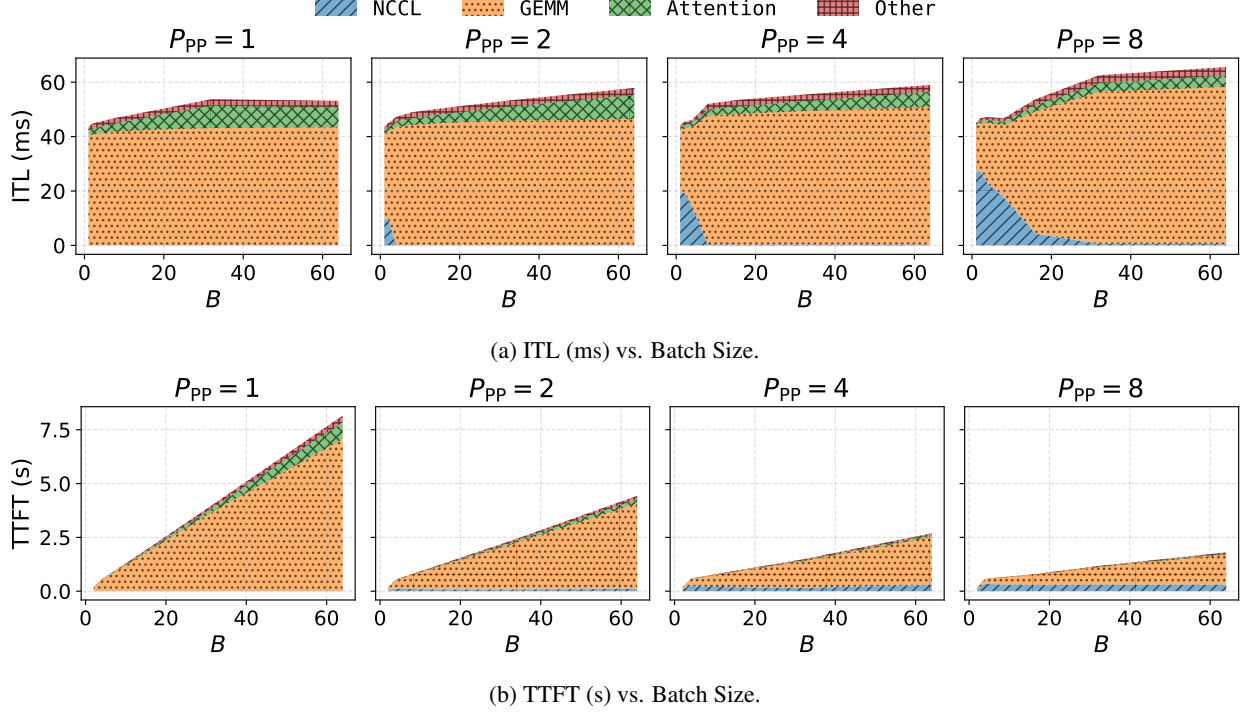


Figure 7: Kernel-level decomposition of (a) ITL and (b) TTFT across different PP degrees for Qwen2.5 - 32B - Instruct. Decode performance is strongly influenced by pipeline bubbles at small batch sizes, whereas prefill workloads maintain high pipeline utilization and are primarily dominated by computation.

ently provides sufficient parallel work to keep deep pipelines occupied, which makes PP particularly well suited for prefill-dominated and long-context inference workloads.

3.4 Hybrid Parallelism: Balancing Communication and Pipeline Bubbles

In this section, we provide experiment results on HB. The results illustrate a complementary trade-off between compute and communication of TP and PP during both prefill and decode. Decoding is dominated by pipeline utilization because only one token is generated per request (i.e., $S = 1$), whereas prefill is dominated by computation and communication due to the long input sequence (i.e., $S = \text{ISL}$). With HB, we would like to understand how much of this trade-off can be balanced out. Throughout the experiments we assume a fixed number of GPU resources (i.e., 8 GPUs), and consider the following 4 parallelism configurations: TP8 representing pure tensor parallelism across GPUs, PP8 for pure pipeline parallelism, PP2TP4 for HB of 2 pipeline stages with 4 GPUs in each stage and PP4TP2 representing 4 pipeline stages with 2 GPUs in each stage.

During decoding, as shown in Fig. 8a, ITL consistently decreases with increasing TP degree because eliminating pipeline bubbles outweighs the additional TP collective communication. We also observe that the PP-dominant configurations (i.e., PP8 and PP4TP2), especially at small batch sizes, the idle latency contributes a noticeable fraction of the ITL. However, bubble gradually disappears as the decode batch size increases. To explain this, we note that during decoding ($S = 1$), the amount of work entering the pipeline is simply $W = B$. Consequently, for small decode batch sizes, the number of work units is insufficient to fully occupy all pipeline stages, resulting in a significant pipeline bubble as shown in (19). As the batch size increases, the amount of work grows proportionally while the fill/drain overhead remains fixed at $P_{PP} - 1$ stages. Therefore, the bubble fraction rapidly decreases, allowing all pipeline stages to remain continuously occupied, which leads to vanishing pipeline bubble as the decode batch size increases. Another important observation in Fig. 8a is that the bubble becomes progressively smaller as the execution shifts from PP-dominant to TP-dominant configurations (i.e., from PP8 to TP8). This behavior follows directly from (18), since the bubble latency is proportional to the pipeline depth. Reducing the P_{PP} decreases the number of fill/drain intervals and therefore shortens the idle latency. In the limiting case of TP8 (i.e., $P_{PP} = 1$), the pipeline consists of a single stage, eliminating fill/drain entirely, resulting in zero pipeline bubble. Consequently, TP-dominant configurations achieve substantially lower ITL during decoding despite incurring additional TP collective communication.

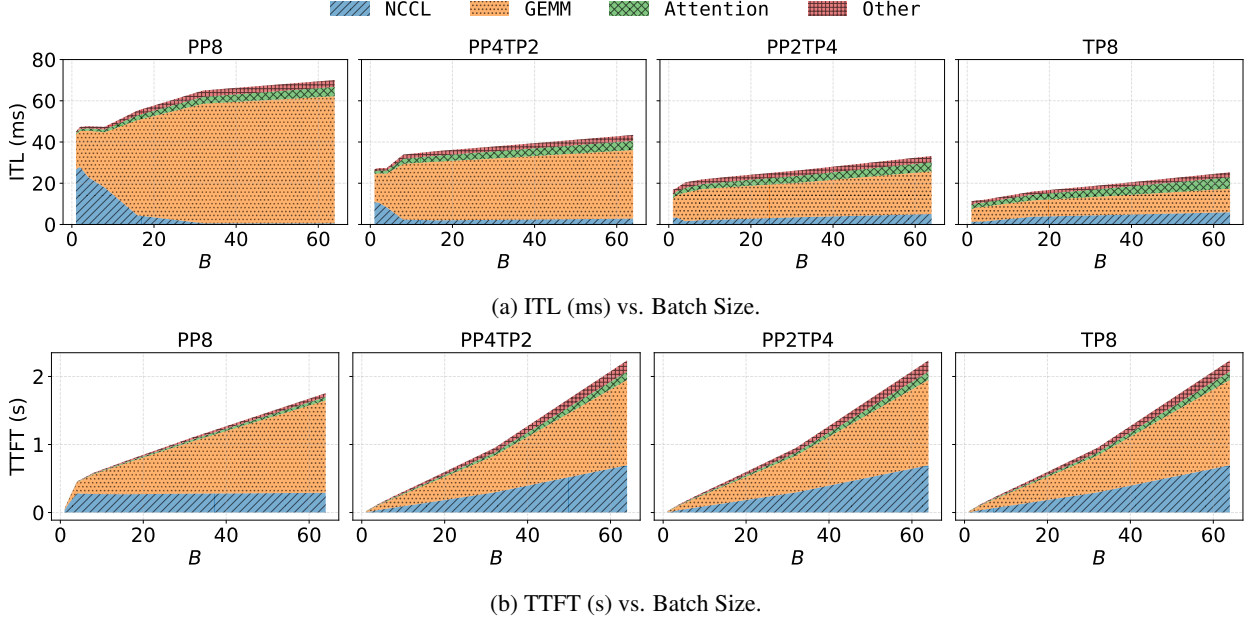
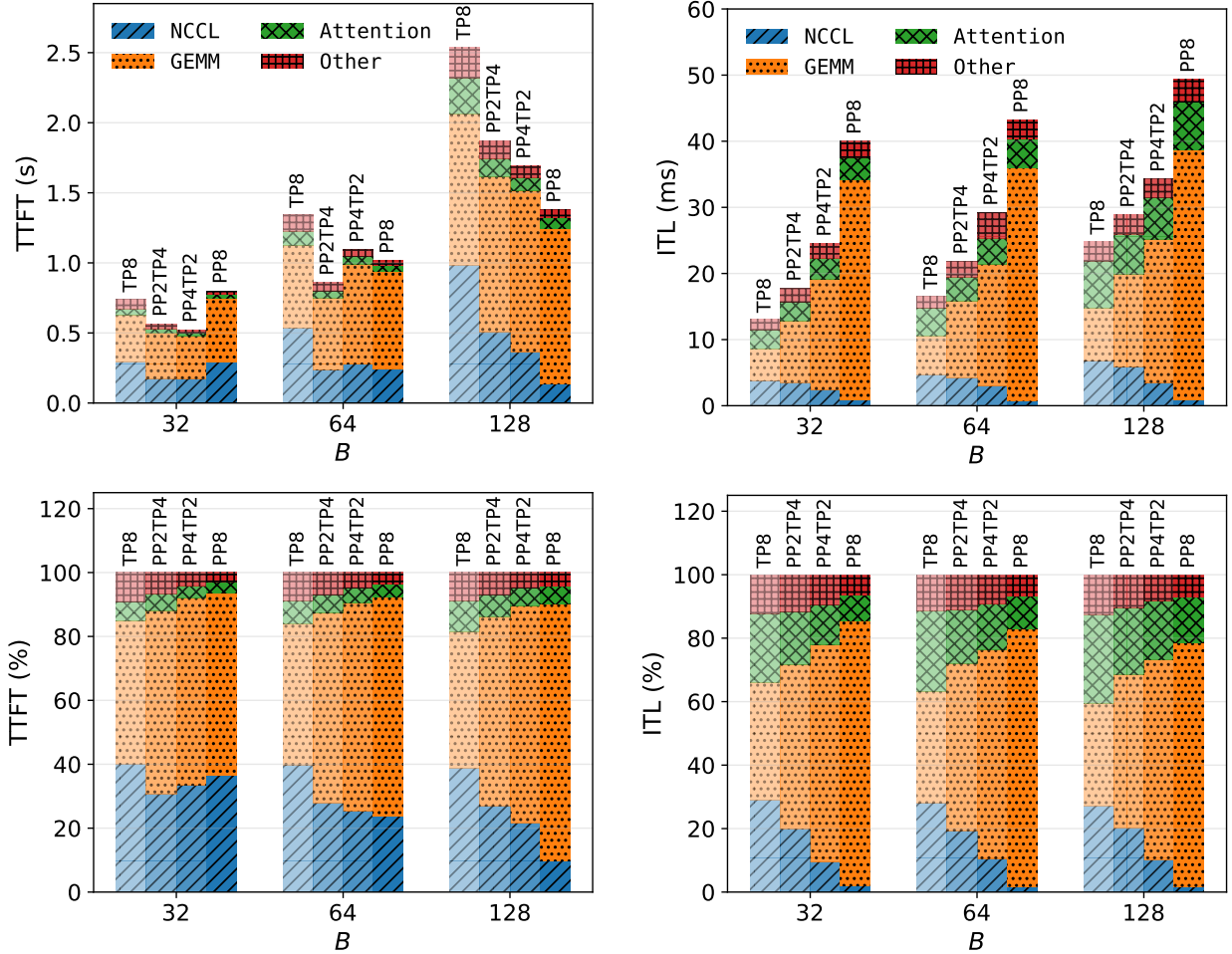


Figure 8: Kernel-level breakdown of (a) ITL during decoding and (b) TTFT during prefill for hybrid TP-PP configurations for Qwen2.5 - 32B - Instruct using a total of 8 GPUs. Decode performance is strongly influenced by PP bubble overhead, which diminishes as execution becomes more TP-dominant, whereas prefill latency is primarily determined by TP communication overhead and useful computation.

Interestingly, during prefill, as shown in Fig 8b, TTFT increases as the execution becomes more TP-dominant. The reason is that TP introduces collective communication after distributed matrix multiplications, where latency grows with the message size. This communication is executed for every transformer layer. Since prefill operates on long sequences ($S = \text{ISL}$), the communicating message size increase linearly with ISL, causing the NCCL component to grow rapidly with batch size. In contrast, PP communication involves point-to-point activation transfers between adjacent stages. This makes PP considerably more communication-efficient for large prefill workloads.

Importantly, we observe in Fig 8b that there is no visible bubble appears in TTFT across different batch sizes, mainly because the behavior of parallelism strategies is fundamentally different from decoding. To explain this we note that during prefill, where all input tokens are processed simultaneously, the amount of work increases to $W = B \times \text{ISL}$, which is typically several orders of magnitude larger than during decoding. According to (19), increasing W dramatically reduces the bubble fraction, for practical input sequence lengths. As a result, the pipeline remains almost fully utilized even for PP-dominant configurations (i.e., PP8 and PP4TP2). Instead, TTFT is almost entirely determined by useful computation and communication rather than pipeline occupancy.

As it is shown in both Fig. 8b and 8a, PP-dominant configurations achieve lower TTFT, whereas TP-dominant configurations provide superior ITL during decoding. These observations highlight the inherent trade-off between PP and TP across prefill and decode. To better understand this trade-off, we plot the breakdown of each kernel latency for TTFT and ITL across different batch sizes and different parallel strategies in Fig. 9a and 9b, respectively. We note that both prefill and decode expose the distinct computational characteristics. Prefill processes the entire input sequence simultaneously, resulting in a large amount of computation per request ($S = \text{ISL}$). Consequently, the execution is predominantly compute-bound, and the overall latency is determined by how efficiently the GPUs sustain large matrix multiplications. In contrast, decoding generates only a single token per request ($S = 1$), significantly reducing the computation performed in each iteration. As a result, decoding becomes increasingly sensitive to pipeline utilization and scheduling overhead, making pipeline bubbles a dominant performance bottleneck. In Fig. 9a, during prefill, PP-dominant configurations consistently achieve the lowest TTFT at $B = 128$ despite using deeper pipelines. This behavior can be explained by the communication characteristics of PP. PP partitions transformer layers across GPUs while keeping the computation of each layer local to a single GPU. Therefore, the only inter-GPU communication is the point-to-point transmission of activations between adjacent stages given in (11), which occurs only once at each pipeline boundary. Since prefill executes long sequences, the computation associated with each stage is substantially larger than the communication cost, allowing communication to be effectively amortized. This observation is consistent with that in Fig. 8b, where the NCCL contribution remains relatively small for PP-dominant configurations even



(a) TTFT vs. Batch Size.

(b) ITL vs. Batch Size.

Figure 9: Trade-off between TP and PP for (a) TTFT during prefill and (b) ITL decode for Qwen2.5-14B-Instruct across 8 GPUs. Prefill performance favors PP due to lower communication overhead, while decode performance favors TP due to reduced pipeline bubbles. Hybrid configurations provide intermediate behavior by balancing TP synchronization costs against PP idle-time overhead.

as the batch size increases. This indicates that most of the execution time is dedicated to useful GEMM rather than communication overhead.

In contrast, TP-dominant configurations exhibit significantly larger TTFT, mainly because of inter-GPU communication after GEMM in every layer. Although TP reduces the message size per GPU, every communication requires global synchronization among all TP ranks. The TP communication latency in (5), is therefore incurred more frequently throughout the forward pass. Since prefill processes long input sequences ($S = \text{ISL}$), the communication volume increases proportionally with S , causing the NCCL overhead to become substantial. This trend is clearly reflected in Fig. 9a, where nearly 40% of the TTFT for TP8 is occupied by NCCL, whereas the PP8 configuration spends only a small fraction of its latency on communication. Consequently, despite eliminating pipeline bubbles, TP suffers from excessive synchronization overhead during prefill, making PP the preferred strategy for maximizing TTFT performance.

The decoding phase exhibits the exact opposite behavior. Since only one token is processed per iteration ($S = 1$), the useful computation per request becomes extremely small, while the pipeline fill/drain overhead remains unchanged. According to (19), PP-dominant configurations experience significant idle time at low and moderate batch sizes, which give rise to substantially higher bubble latency. Eliminating these bubbles through TP produces a much larger latency reduction than the additional NCCL can offset. This behavior is clearly shown in Fig. 9b, where the PP8 configuration exhibits the highest ITL despite having the lowest communication overhead. As the pipeline depth decreases (from PP8 to TP8), the number of fill/drain intervals decreases proportionally, eventually disappearing entirely when $P_{\text{PP}} = 1$ (i.e., TP8). Consequently, TP8 achieves the lowest ITL despite the highest NCCL among all configurations.

Overall, PP and TP optimize two fundamentally different bottlenecks. PP minimizes synchronization overhead and is therefore ideally suited for the compute-intensive prefill phase, whereas TP minimizes pipeline occupancy loss by eliminating fill/drain overhead and is therefore more effective for latency-critical decoding. The hybrid configurations (PP4TP2 and PP2TP4) naturally provide intermediate behavior, balancing TP communication overhead against PP bubble latency. They offer a compromise between TTFT and ITL rather than optimizing either metric individually. This observation is consistent with the analytical latency model we developed in Section 2, where PP communication, TP communication, and pipeline bubble latency contribute independently to the total inference latency.

4 Conclusion

Distributed execution has become essential for serving modern LLMs, yet understanding the performance trade-offs among TP, PP, and HB remains a fundamental challenge. In this paper, we presented a unified analytical framework for modeling distributed LLM inference by decomposing end-to-end latency into computation, inter-GPU communication, and pipeline bubble overhead. We derived analytical models for TP collective communication, PP point-to-point communication, and pipeline utilization, capturing their dependence on batch size, sequence length, hidden dimension, message size, and parallelism degree. Unlike existing empirical studies, our framework provides a principled explanation of how these factors jointly determine inference performance across both the prefill and decoding phases.

Our analysis and experimental evaluation demonstrate that the proposed framework accurately captures the contrasting behavior of different parallelism strategies, explaining why PP is more effective for the compute-intensive prefill phase, whereas TP achieves lower decoding latency by eliminating pipeline bubbles despite higher communication overhead. By providing a unified understanding of the compute, communication and bubble latency trade-offs, the proposed model enables systematic evaluation of parallelism strategies across diverse models, workloads, and hardware configurations. We believe this framework provides a practical foundation for performance modeling, capacity planning, and the design of future distributed LLM serving systems that can adapt parallelism strategies to evolving workload characteristics.

Acknowledgments

The authors would like to acknowledge the support of the Dell leadership team and thank colleagues across Dell for their valuable discussions, constructive feedback, and technical insights that contributed to this research.

References

- [1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, “Taming throughput-latency tradeoff in LLM inference with sarathi-serve,” in *18th USENIX symp. on operating sys. design and implementation (OSDI 24)*, pp. 117–134, 2024.
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with paged attention,” in *Pro. of the 29th symp. on operating systems principles*, pp. 611–626, 2023.
- [3] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training multi-billion parameter language models using model parallelism,” 2019. Available: <https://arxiv.org/abs/1909.08053>.
- [4] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. Le, Y. Wu, *et al.*, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” *In Proc. Advances neural info. process. sys.*, vol. 32, 2019.
- [5] A. Harlap, D. Narayanan, A. Phanishayee, V. Seshadri, N. Devanur, G. Ganger, and P. Gibbons, “Pipedream: Fast and efficient pipeline parallel DNN training,” 2018. Available: <https://arxiv.org/abs/1806.03377>.
- [6] D. Narayanan, M. Shoenybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, *et al.*, “Efficient large-scale language model training on GPU clusters using Megatron-LM,” *In Proc. of the Int. conf. for high performance computing, networking, storage and analysis*, pp. 1–15, 2021.
- [7] R. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, *et al.*, “Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale,” in *SC22: Int. Conf. for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, IEEE, 2022.
- [8] NVIDIA Corporation, “TensorRT-LLM documentation.” <https://docs.nvidia.com/tensorrt-llm/index.html>, 2026. Accessed: 2026-07-13.

- [9] vLLM Project Contributors, “vLLM documentation.” <https://docs.vllm.ai/en/stable/>, 2026. Accessed: 2026-07-13.
- [10] NVIDIA Corporation, “Nvidia dynamo.” <https://www.nvidia.com/en-us/ai/dynamo/>, 2026. Accessed: 2026-07-13.
- [11] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” *18th USENIX Symp. Operating Sys. Design and Implementation (OSDI 24)*, pp. 193–210, 2024.
- [12] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *2024 ACM/IEEE 51st Annual Int. Symp. on Computer Architecture (ISCA)*, pp. 118–132, IEEE, 2024.
- [13] W. Fedus and N. Zoph, B. and Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [14] R. W. Hockney, “The communication challenge for MPP: Intel paragon and meiko CS-2,” *Parallel Comput.*, vol. 20, p. 389–398, Mar. 1994.
- [15] G. Iannello, M. Lauria, and S. Mercolino, “LogP performance characterization of fast messages atop myrinet,” in *Proc. Sixth Euromicro Workshop on Parallel and Distributed Process.*, pp. 395–401, 1998.
- [16] NVIDIA Corporation, “NVIDIA Collective Communications Library (NCCL).” <https://developer.nvidia.com/nccl>, 2026. Accessed: 2026-07-10.
- [17] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, p. 65–76, Apr. 2009.
- [18] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, *et al.*, “Mooncake: A KV cache-centric disaggregated architecture for LLM serving,” *ACM Transactions on Storage*, 2024.
- [19] Z. Hu, S. Shen, T. Bonato, S. Jeaugey, C. Alexander, E. Spada, J. Dinan, J. Hammond, and T. Hoefler, “Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms,” in *2025 IEEE Symp. on High-Performance Interconnects (HOTI)*, pp. 48–59, IEEE, 2025.
- [20] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” 2025. Available: <https://arxiv.org/abs/2412.15115>.
- [21] NVIDIA, “NVIDIA A100 tensor core GPU: Unprecedented acceleration at every scale.” <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, 2026. Accessed: 2026-09-25.
- [22] vLLM v0.15.0, “vLLM: Easy, fast, and cheap llm serving for everyone.” <https://docs.vllm.ai/en/v0.15.0/>, 2026. Accessed: 2026-09-25.
- [23] “NVIDIA Nsight Systems v2026.5.1 .” <https://developer.nvidia.com/nsight-systems>, 2026. Accessed: 2026-09-25.